



第7章 动态规划

Dynamic Programming

Dynamic Programming is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems.

《算法导论》第15章

斐波那契数列

第1个月：有1对兔子(1)，没有繁殖能力，1+0共有1对兔子-> 第1个月 (1对)：

第2个月：兔子(1)进入成熟期，1+0共有1对兔子-> 第2个月 (1对)：

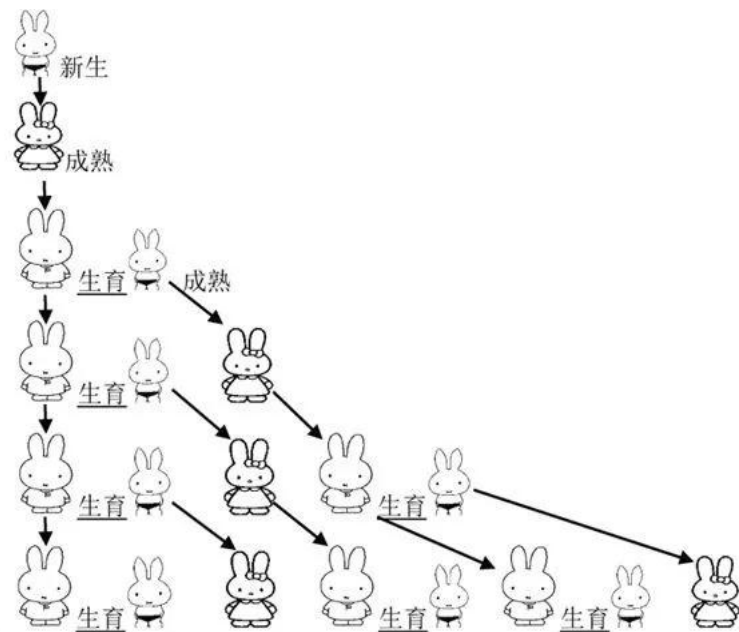
第3个月：兔子生了1对小兔子(2)，1+1共有2对兔子-> 第3个月 (2对)：

第4个月：(1)又生了兔子(3)，1+2共有3对兔子-> 第4个月 (3对)：

第5个月：(1)生了(4)，(2)生了(5)，2+3共有5对兔子-> 第5个月 (5对)：

第6个月：(1)(2)(3)各生下1对小兔子，3+5共有8对兔子-> 第6个月 (8对)：

.....



每个月兔子数的数列是：

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

因此：

$$\begin{aligned} f(1) &= 1, f(2) = 1, \\ f(3) &= f(1) + f(2), \\ f(4) &= f(3) + f(2), \dots \end{aligned}$$

$$F(n) = \begin{cases} 1 & , n = 1 \\ 1 & , n = 2 \\ F(n-1) + F(n-2) & , n > 2 \end{cases}$$

斐波那契数列 (Top-down)

$$F(n) = \begin{cases} 1 & , n = 1 \\ 1 & , n = 2 \\ F(n-1) + F(n-2) & , n > 2 \end{cases}$$

如何设计算法，编写程序，给定任意的 n 求 $F(n)$

```
1. public int fib(int n) {  
2.     if (n < 1) {  
3.         return -1;  
7.     }  
5.     if (n == 1 || n == 2) {  
6.         return 1;  
7.     }  
8.     return fib(n - 1) + fib(n - 2);  
9. }
```

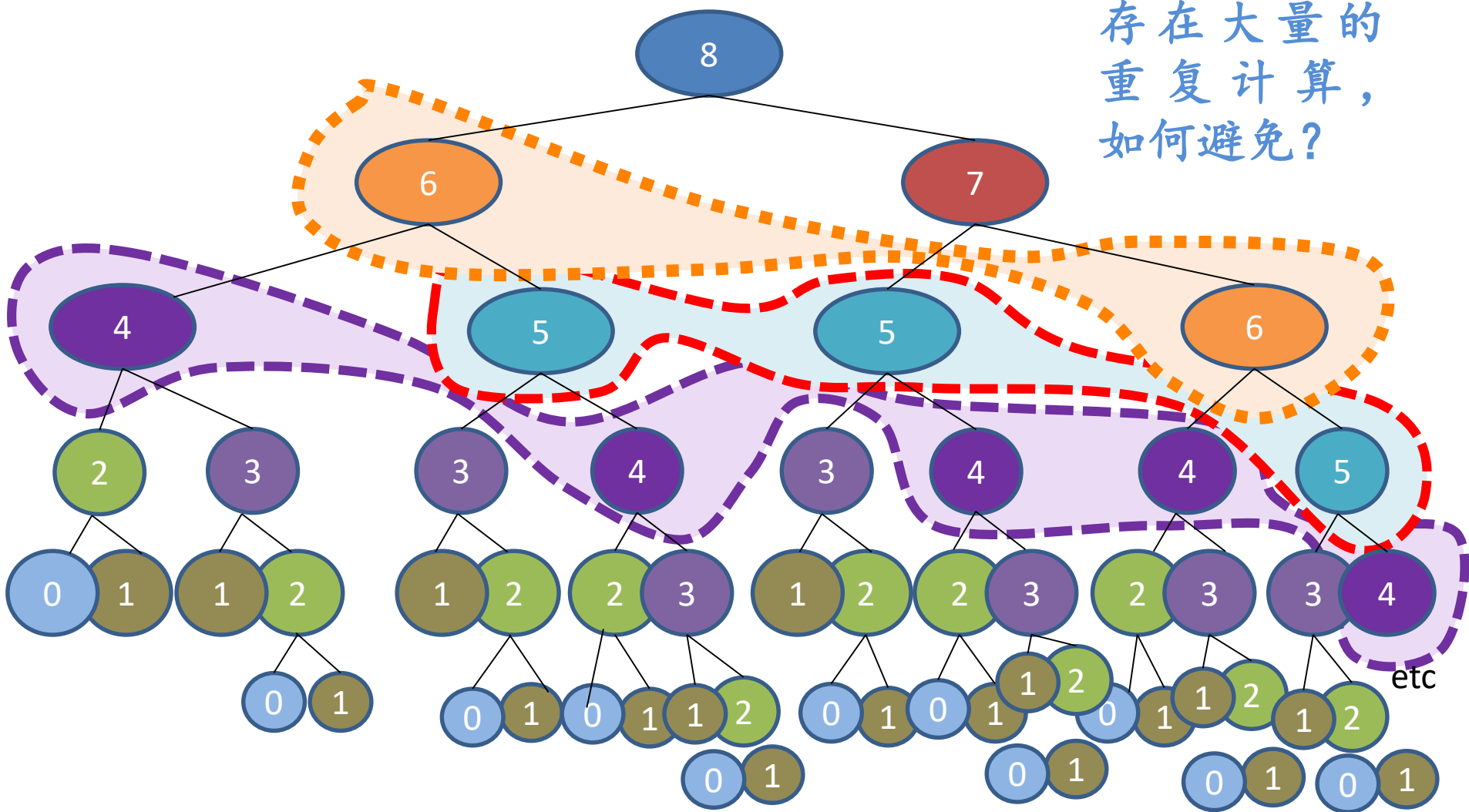
时间复杂度: $O(2^n)$



Can we do better?

斐波那契数列

存在大量的
重复计算，
如何避免？



本讲内容

7.1 动态规划的原理

7.2 矩阵乘法问题

7.3 最长公共子序列

7.4 0-1背包问题

7.5 最优二叉搜索树

动态规划 = 分治递归 + 记忆存储

- 动态规划是一种算法设计的范式（paradigm）或思想
不是解决某一具体问题的具体算法
- 理查德·贝尔曼（Richard Bellman）在1950年代首次提出了这个名字，当时他正为美国兰德公司工作，主要为美国空军和政府项目服务。

“It’s impossible to use the word, dynamic, in the pejorative sense ... I thought dynamic programming was a good name. It was something not even a Congressman could object to.”

——理查德·贝尔曼



理查德·贝尔曼（1920年8月26日—1984年3月19日），美国应用数学家，美国国家科学院院士，动态规划的创始人。

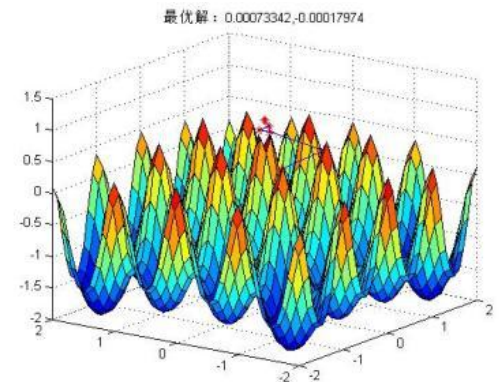
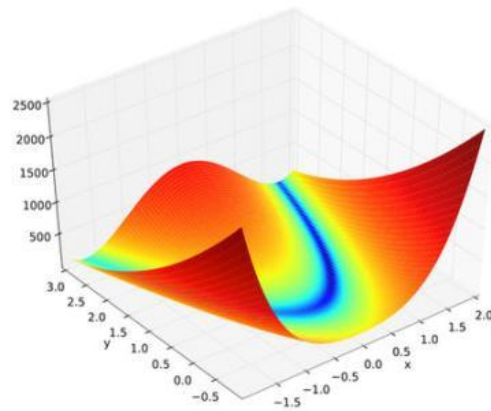
Why?

“那些遗忘过去的人注定要重蹈覆辙。”——乔治·桑塔亚纳

- 分治技术（无记忆递归）
 - 子问题是相互独立的
 - 如果子问题不是相互独立的，分治方法将重复计算公共子问题，效率很低

What?

- 动态规划的特点（有记忆迭代）
 - 把原始问题划分成一系列子问题
 - 求解每个子问题仅一次，并将其结果保存在一个表中，以后用到时直接存取，不重复计算，节省计算时间
 - 自底向上地计算
- 适用范围（优化问题）
 - 优化问题给定一组约束条件和一个代价函数，在解空间中搜索具有最小或最大代价的优化解
 - 很多优化问题可分为多个子问题，子问题相互关联，子问题的解被重复使用



How?

- 使用动态规划的条件
 - 优化子结构(正确性)
 - 当一个问题的高效解包含了子问题的高效解时，我们称这个问题具有优化子结构。
 - 缩小子问题集合，只需那些优化问题中包含的子问题，降低实现复杂性
 - 优化子结构使得我们能自下而上地完成求解过程
 - 重叠子问题(有效性)
 - 在问题的求解过程中，很多子问题的解将被多次使用

- 动态规划算法的设计步骤

- ① 分析优化解的结构

- ② 递归地定义最优值的代价

- ③ 自底向上地计算优化解的代价保存之，并获取构造最优值的信息

- ④ 根据构造最优值的信息构造优化解

- 步骤①-③是动态规划的基本步骤。在只需要计算出最优值的情形，步骤④将被省略。

- 若求出原问题的一个优化解，则必须执行步骤④。需要通过步骤③中记录信息得到优化解。

本讲内容

7.1 动态规划的原理

7.2 矩阵乘法问题

7.3 最长公共子序列

7.4 0-1背包问题

7.5 最优二叉搜索树

问题的定义

- 输入: $\langle A_1, A_2, \dots, A_n \rangle$, $A_i \in \mathbb{R}^{p_{i-1} \times p_i}$ 是矩阵
- 输出: 计算 $A_1 \times A_2 \times \dots \times A_n$ 的最小代价方法

矩阵乘法的代价/复杂性: 乘法的次数

若 A 是 $p \times q$ 矩阵, B 是 $q \times r$ 矩阵, 则 $A \times B$ 的代价是 $O(pqr)$

动机

- 矩阵链乘法的实现

- 矩阵乘法满足结合率。
- 计算一个矩阵链的乘法可有多种方法：

例如,

$$\begin{aligned} & A_1 \times A_2 \times A_3 \times A_4 \\ &= \left(A_1 \times \left(A_2 \times \left(A_3 \times A_4 \right) \right) \right) \\ &= \left(\left(A_1 \times A_2 \right) \times \left(A_3 \times A_4 \right) \right) \\ &= \left(\left(\left(A_1 \times A_2 \right) \times A_3 \right) \times A_4 \right) \\ &= \left(A_1 \times \left(\left(A_2 \times A_3 \right) \times A_4 \right) \right) \\ &= \left(\left(A_1 \times \left(A_2 \times A_3 \right) \right) \times A_4 \right) \end{aligned}$$

- 矩阵链乘法的代价与计算顺序的关系

– 设 $A_1 = 10 \times 100$ 矩阵, $A_2 = 100 \times 5$ 矩阵,

$A_3 = 5 \times 50$ 矩阵

$$T((A_1 \times A_2) \times A_3) = 10 \times 100 \times 5 + 10 \times 5 \times 50 \\ = 7500$$

$$T(A_1 \times (A_2 \times A_3)) = 100 \times 5 \times 50 + 10 \times 100 \times 50 \\ = 75000$$

结论: 不同计算顺序有不同的代价

- 矩阵链乘法优化问题的解空间(穷举法)

- 设 $p(n)$ = 计算 n 个矩阵乘积的方法数

- $p(n)$ 的递归方程

$$(A_1 \times \cdots \times A_k) \times (A_{k+1} \times \cdots \times A_n)$$

$$p(n) = \begin{cases} 1, & n = 1 \\ \sum_{k=1}^{n-1} p(k)p(n-k), & n > 1 \end{cases}$$

$$\begin{aligned} p(n) &= C(n-1) = \text{Catalan数} = \frac{1}{n} \binom{2(n-1)}{n-1} = \frac{(2(n-1))!}{n!(n-1)!} \\ &= \Omega(4^n/n^{3/2}) \end{aligned}$$

如此之大的解空间是无法用枚举方法求出最优解的！

矩阵链乘法问题的动态规划算法

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

第一步：分析优化解的结构

- 两个记号

- $A_{i \sim j} = A_i \times A_{i+1} \times \cdots \times A_j$

- $\text{cost}(A_{i \sim j}) = \text{计算 } A_{i \sim j} \text{ 的代价}$

第一步：分析优化解的结构

- 优化解的结构（任何最优解都是由子问题的最优解构成）
 - 若计算 $A_{1\sim n}$ 的优化顺序在 k 处断开矩阵链，即 $A_{1\sim n}^1 = A_{1\sim k}^1 \times A_{k+1\sim n}^1$ ，则在 $A_{1\sim n}^1$ 的优化顺序中，对应于子问题 $A_{1\sim k}^1$ 的解必定是 $A_{1\sim k}$ 的优化解；
 - 对应于子问题 $A_{k+1\sim n}^1$ 的解必须是 $A_{k+1\sim n}$ 的优化解。

证明：(反证法)

假设最优矩阵链乘 $A_{1\sim n}^1 = (A_1 \times \cdots \times A_k) \times (A_{k+1} \times \cdots \times A_n)$

假设 $A_{1\sim k}^1 = A_1 \times (A_2 \times A_3) \times \cdots \times A_k$

$A_{k+1\sim n}^1 = A_{k+1} \times \cdots \times (A_{n-1} \times A_n)$

反证法：假设有一个 $A_{1\sim k}^2 = A_1 \times \cdots \times (A_{k-1} \times A_k)$ 比 $A_{1\sim k}^1$ 更优，那么 $A_{1\sim n}^2 = A_{1\sim k}^2 \times A_{k+1\sim n}^1$ 将构成 $A_{1\sim n}$ 的一个新矩阵链乘。显然 $A_{1\sim n}^2$ 的计算代价比 $A_{1\sim n}^1$ 少，这与 $A_{1\sim n}^1$ 是 $A_{1\sim n}$ 的最优矩阵链乘相矛盾。

具有优化子结构：问题的优化解包括子问题优化解

- 子问题重叠性

$$A_1 \times A_2 \times A_3 \times A_4$$

$$(A_1) \times (A_2 \times A_3 \times A_4)$$

$$(A_1 \times A_2) \times (A_3 \times A_4)$$

$$(A_1 \times A_2 \times A_3) \times (A_4)$$

$$(A_2 \times A_3)$$

$$(A_3 \times A_4)$$

$$(A_1 \times A_2)$$

$$(A_3 \times A_4)$$

$$(A_1 \times A_2)$$

$$(A_2 \times A_3)$$

具有子问题重叠性

第二步：递归地定义最优值的代价

- 假设

- $m[i, j]$ = 计算 $A_{i \sim j}$ 的最小乘法数
- $m[1, n]$ = 计算 $A_{1 \sim n}$ 的最小乘法数
- $A_1 \cdots A_k A_{k+1} \cdots A_n$ 是优化解 (k 实际上是不可预知)

- 代价方程

$$m[i, i] = \text{计算 } A_{i \sim i} \text{ 的最小乘法数} = 0$$

$$m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j\}$$

其中, $p_{i-1} p_k p_j$ 是计算 $A_{i \sim k} \times A_{k+1 \sim j}$ 所需乘法数, $A_{i \sim k}$ 和 $A_{k+1 \sim j}$ 分别是 $p_{i-1} \times p_k$ 和 $p_k \times p_j$ 矩阵。

考虑到所有的 k ，优化解的代价方程为

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

代价矩阵 m 的维度是？

代价矩阵 m 是上三角？

第三步：自底向上计算优化解的代价

$$m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\}$$

m[1,1]

m[1,2]

m[1,3]

m[1,4]

m[1,5]

m[2,2]

m[2,3]

m[2,4]

m[2,5]

m[3,3]

m[3,4]

m[3,5]

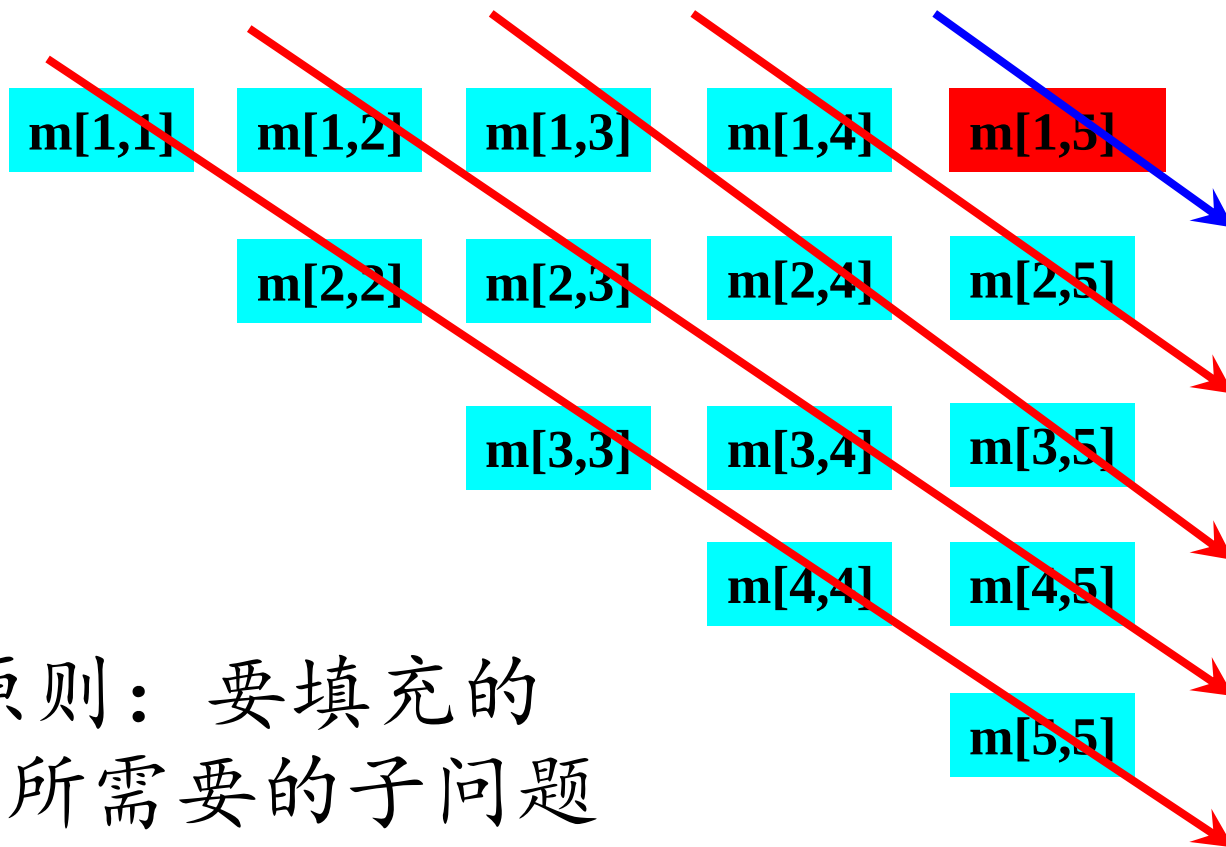
m[4,4]

m[4,5]

m[5,5]

$$m[1,5] = \min \begin{cases} m[1,1] + m[2,5] + p_0p_1p_5 & k = 1 \\ m[1,2] + m[3,5] + p_0p_2p_5 & k = 2 \\ m[1,3] + m[4,5] + p_0p_3p_5 & k = 3 \\ m[1,4] + m[5,5] + p_0p_4p_5 & k = 4 \end{cases}$$

$$m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\}$$



填充原则：要填充的
 $m[i, j]$ 所需要的子问题的
 代价都是已知的

算法

Matrix-Chain-Order

Input: 向量 p , p 中存放各个矩阵的行数列数

Output: m 完成矩阵链乘的最小乘法数

过程: Matrix-Chain-Order(p)

```
1.  $n \leftarrow \text{length}(p) - 1;$  /* $p_0$ 到 $p_n$ , 所以要减1*/  
2. FOR  $i \leftarrow 1$  TO  $n$  DO  
3.      $m[i, i] \leftarrow 0;$   
4. FOR  $l \leftarrow 2$  TO  $n$  DO /*计算第 $l$ 个对角线*/  
5.     FOR  $i \leftarrow 1$  TO  $n - l + 1$  DO  
6.          $j \leftarrow i + l - 1;$   
7.          $m[i, j] \leftarrow \infty;$   
8.         FOR  $k \leftarrow i$  TO  $j - 1$  DO /* 计算 $m[i, j]$  */  
9.              $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j;$   
10.            IF  $q < m[i, j]$  THEN  $m[i, j] \leftarrow q;$   
11. Return  $m.$ 
```

第四步：获取构造最优值的信息

Matrix-Chain-Order

Input: 向量 p , p 中存放各个矩阵的行数列数

Output: m 完成矩阵链乘的最小乘法数

过程: Matrix-Chain-Order(p)

$S[i, j]$ 记录 $A_i A_{i+1} \cdots A_j$ 的最优划分处

```
1.  $n \leftarrow \text{length}(p) - 1;$ 
2. FOR  $i \leftarrow 1$  TO  $n$  DO
3.      $m[i, i] \leftarrow 0, S[i, i] \leftarrow 0;$  /*初始化主对角线*/
4. FOR  $l \leftarrow 2$  TO  $n$  DO /*计算第 $l$ 对角线 */
5.     FOR  $i \leftarrow 1$  TO  $n - l + 1$  DO
6.          $j \leftarrow i + l - 1;$ 
7.          $m[i, j] \leftarrow \infty;$ 
8.         FOR  $k \leftarrow i$  TO  $j - 1$  DO /* 计算 $m[i, j]$  */
9.              $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j;$ 
10.            IF  $q < m[i, j]$  THEN  $m[i, j] \leftarrow q, S[i, j] = k;$ 
11. Return  $m, S.$ 
```

构造最优解

Print-Optimal-Parens(S, i, j)

1. IF $j = i$
2. THEN Print "A_i";
3. ELSE Print "("
4. Print-Optimal-Parens($S, i, S[i, j]$)
5. Print-Optimal-Parens($S, S[i, j]+1, j$)
6. Print ")"

$S[i, j]$ 记录 $A_i \cdots A_j$ 的最优划分处;
 $S[i, S[i, j]]$ 记录 $A_i \cdots A_{S[i, j]}$ 的最优划分处;
 $S[S[i, j] + 1, j]$ 记录 $A_{S[i, j]+1} \cdots A_j$ 的最优划分处。

调用 **Print-Optimal-Parens($S, 1, n$)**

即可输出 $A_{1 \sim n}$ 的优化计算顺序

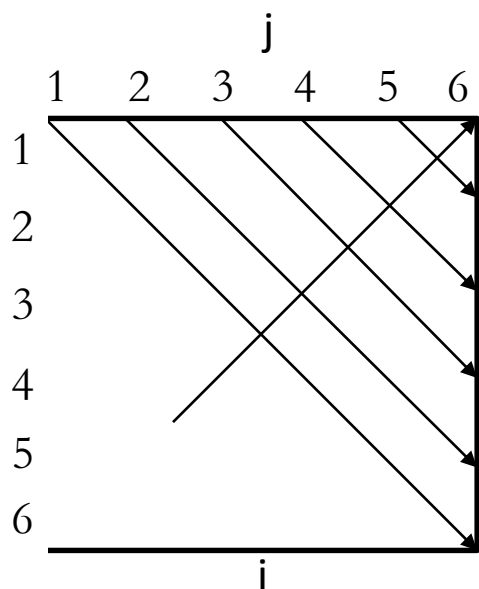
算法复杂性

- 时间复杂性
 - 计算代价的时间
 - (l, i, k) 三层循环, 每层至多 $n - 1$ 步
 - $O(n^3)$
 - 构造最优解的时间: $O(n)$
 - 总时间复杂性为: $O(n^3)$
- 空间复杂性
 - 使用数组 p, m 和 S
 - 需要空间 $O(n^2)$

示例

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

$$m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\}$$



(a) 计算次序

	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

(b) $m[i, j]$

	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

(c) $S[i, j]$

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 & k = 2 \\ m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 & k = 3 \\ m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 & k = 4 \end{cases}$$

$$(A_1 \times (A_2 \times A_3)) \times ((A_4 \times A_5) \times A_6)$$

本讲内容

7.1 动态规划的原理

7.2 矩阵乘法问题

7.3 最长公共子序列

7.4 0-1背包问题

7.5 最优二叉搜索树

这两个物种有多相似



DNA:

AGCCCTAAGGGCTACCTAGCTT



DNA:

GACAGCCTACAAGCGTTAGCTTG

最长公共子序列



DNA:

AGCCCTAAGGGCTACCTAGCTT



DNA:

GACAGCCTACAAGCGTTAGCTTG

AGCCTAAGCTTAGCTT

最长公共子序列 问题的定义

- 子序列 (Subsequence)
 - $X = (A, B, C, B, D, B)$
 - $Z = (B, C, D, B)$ 是 X 的子序列
 - $W = (B, D, A)$ 不是 X 的子序列
- 公共子序列 (Common Subsequence)
 - Z 是序列 X 与 Y 的公共子序列, 如果 Z 是 X 的子序也是 Y 的子序列。

问题的定义

最长公共子序列 (*LCS*) 问题:

输入: $X = (x_1, x_2, \dots, x_n)$

$Y = (y_1, y_2, \dots, y_m)$

输出: $Z = X$ 与 Y 的最长公共子序列

- 利用暴力搜索方法求解LCS问题, 即穷举 X 所有的子序列, 检查每个子序列是否为 Y 的子序列。
- 序列 X 的每个子序列的下标集合为 $\{1, 2, \dots, n\}$ 的子集, 所以共有 2^n 个子序列。
- 暴力搜索方法的时间复杂度是指数阶的。

最长公共子序列结构分析

- 第 i 前缀

- 设 $X = (x_1, x_2, \dots, x_n)$ 是一个序列, X 的第 i 前缀 X_i 是一个序列, 定义为 $X_i = (x_1, x_2, \dots, x_i)$

例. $X = (A, B, D, C, A)$, $X_1 = (A)$, $X_2 = (A, B)$, $X_3 = (A, B, D)$

使用动态规划

- 分析优化解的结构 
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

第一步：分析优化解的结构

定理（优化子结构）： 设 $X = (x_1, \dots, x_m)$ 、 $Y = (y_1, \dots, y_n)$ 是两个序列， $Z = (z_1, \dots, z_k)$ 是 X 与 Y 的 LCS ，我们有：

(1) 如果 $x_m = y_n$ ，则 $z_k = x_m = y_n$ ， Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的 LCS ，即， $LCS_{XY} = LCS_{X_{m-1}Y_{n-1}} + \langle x_m = y_n \rangle$ ；

(2) 如果 $x_m \neq y_n$ ，且 $z_k \neq x_m$ ，则 Z 是 X_{m-1} 和 Y 的 LCS ，即 $LCS_{XY} = LCS_{X_{m-1}Y}$ ；

(3) 如果 $x_m \neq y_n$ ，且 $z_k \neq y_n$ ，则 Z 是 X 与 Y_{n-1} 的 LCS ，即 $LCS_{XY} = LCS_{XY_{n-1}}$ 。

(1) $X = (x_1, \dots, x_{m-1}, x_m)$, $Y = (y_1, \dots, y_{n-1}, x_m)$, $Z = (z_1, \dots, z_k)$ 是 X 与 Y 的 LCS , 则

$$LCS_{XY} = LCS_{X_{m-1}Y_{n-1}} + \langle x_m = y_n \rangle$$

证明: (反证法)

- 设 $z_k \neq x_m$, 则可加 $x_m = y_n$ 追加到 Z 的末尾, 得到一个长为 $k+1$ 的 X 与 Y 的公共序列, 与 Z 是 X 和 Y 的 LCS 矛盾, 于是 $z_k = x_m = y_n$ 。
- 现在证明 Z_{k-1} 是 X_{m-1} 与 Y_{n-1} 的 LCS 。显然 Z_{k-1} 是 X_{m-1} 与 Y_{n-1} 的公共序列。我们需要证明 Z_{k-1} 是 LCS 。设不然, 则存在 X_{m-1} 与 Y_{n-1} 的公共子序列 W , W 的长大于 $k-1$ 。增加 $x_m = y_n$ 到 W , 我们得到一个长大于 k 的 X 与 Y 的公共序列, 与 Z 是 X 和 Y 的 LCS 矛盾。于是 Z_{k-1} 是 X_{m-1} 与 Y_{n-1} 的 LCS 。

(2) $X = (x_1, \dots, x_{m-1}, x_m), Y = (y_1, \dots, y_{n-1}, y_n),$

$x_m \neq y_n$, 且 $z_k \neq x_m$, 则 $LCS_{XY} = LCS_{X_{m-1}Y}$

证明: (反证法)

由于 $z_k \neq x_m$, Z 是 X_{m-1} 与 Y 的公共子序列。我们来证 Z 是 X_{m-1} 与 Y 的 LCS 。设 X_{m-1} 与 Y 有一个公共子序列 W , W 的长度大于 k , 则 W 也是 X 与 Y 的公共子序列, 与 Z 是 LCS 矛盾。

(3) 同 (2) 可证。

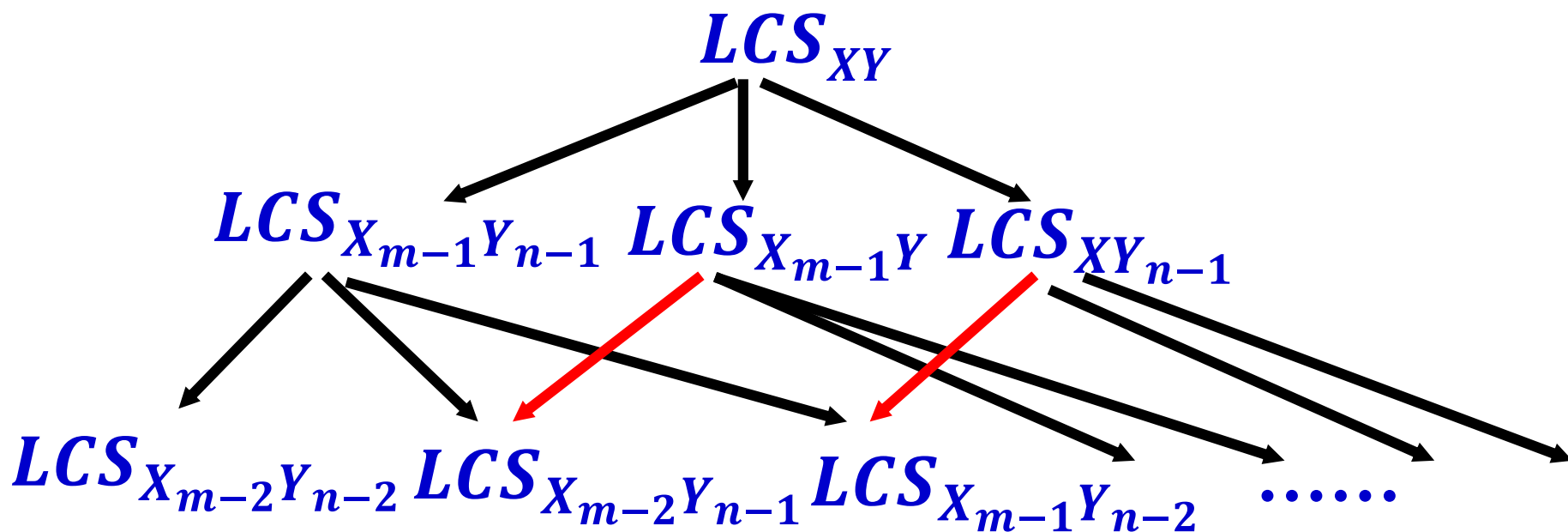
定理1表明: 两个序列的LCS包含两个序列前缀的LCS。

因此, LCS问题具有优化子结构。可以考虑动态规划。

X和***Y***的LCS的优化解结构为

$$LCS_{XY} = \begin{cases} LCS_{X_{m-1}Y_{n-1}} + \langle x_m = y_n \rangle & \text{if } x_m = y_n \\ LCS_{X_{m-1}Y} & \text{if } x_m \neq y_n, z_k \neq x_m \\ LCS_{XY_{n-1}} & \text{if } x_m \neq y_n, z_k \neq y_n \end{cases}$$

- 子问题重叠性



LCS 问题具有子问题重叠性

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价 
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

第二步：递归地定义最优值的代价

建立LCS长度的递归方程

- $C[i, j]$ = 前缀 X_i 与前缀 Y_j 的LCS的长度
- LCS长度的递归方程

$$C[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ 或 } j = 0 \\ C[i - 1, j - 1] + 1 & \text{if } i, j > 0 \text{ 且 } x_i = y_j \\ \text{Max}(C[i, j - 1], C[i - 1, j]) & \text{if } i, j > 0 \text{ 且 } x_i \neq y_j \end{cases}$$

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解



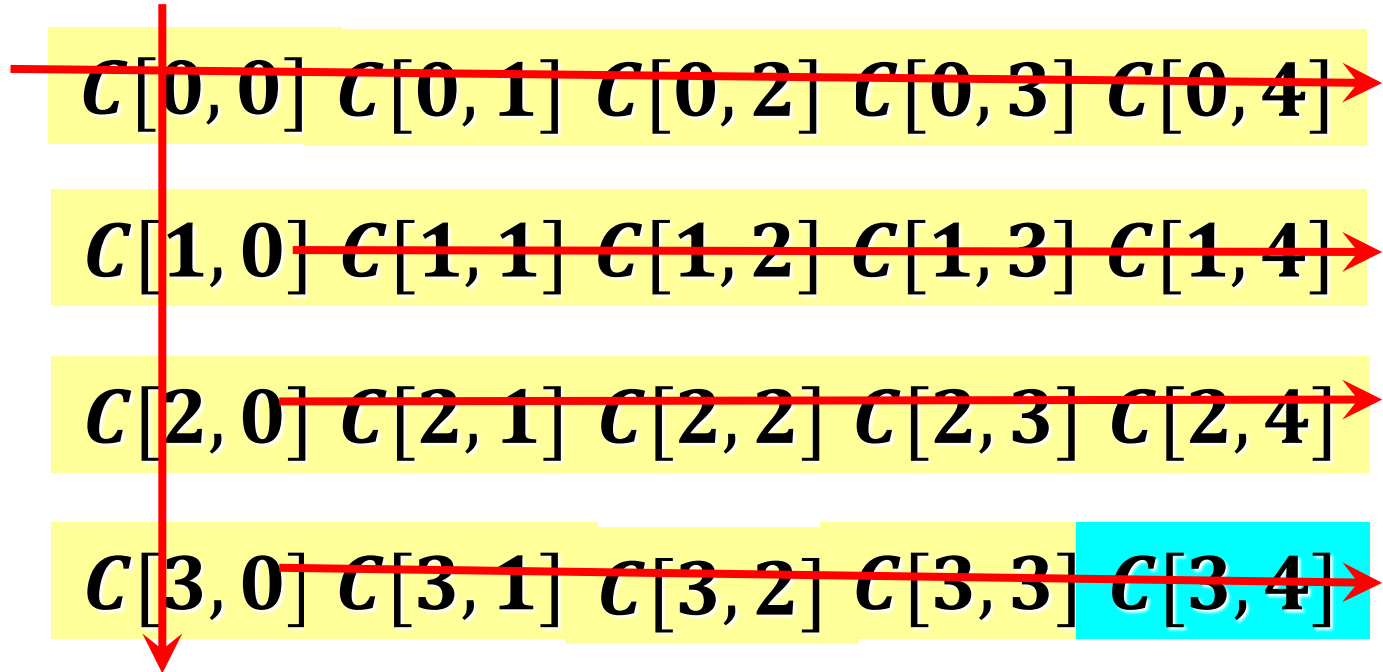
第三步：自底向上计算LCS的长度

- 基本思想

	$C[i-1, j-1]$	$C[i-1, j]$	
	$C[i, j-1]$	$C[i, j]$	

$$C[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ 或 } j = 0 \\ C[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ 且 } x_i = y_j \\ \text{Max}(C[i, j-1], C[i-1, j]) & \text{if } i, j > 0 \text{ 且 } x_i \neq y_j \end{cases}$$

- 计算过程



填充原则：要填充的 $C[i,j]$ 所需要的子问题的代价都是已知的

- 计算LCS长度的算法

- 数据结构

$C[0:m, 0:n]$: $C[i, j]$ 是 X_i 与 Y_j 的 *LCS* 的长度

$B[1:m, 1:n]$: $B[i, j]$ 是指针, 指向计算 $C[i, j]$ 时所选择的子问题的优化解所对应的 C 表的表项

算法

LCS-length(X,Y)

Input: 序列X和Y

Output: C最长公共子序列长度, B构造优化解信息

过程: LCS-length(X, Y)

1. $m \leftarrow \text{length}(X)$; $n \leftarrow \text{length}(Y)$;

2. For $i \leftarrow 1$ To m Do $C[i,0] \leftarrow 0$; /*初始化第0列*/

3. For $j \leftarrow 1$ To n Do $C[0,j] \leftarrow 0$; /*初始化第0行*/

4. For $i \leftarrow 1$ To m Do

5. For $j \leftarrow 1$ To n Do

6. IF $x_i = y_j$

7. Then $C[i,j] \leftarrow C[i-1,j-1] + 1$; $B[i,j] \leftarrow \text{"\textasciitilde{~}"}$;

8. Else IF $C[i-1,j] \geq C[i,j-1]$

9. Then $C[i,j] \leftarrow C[i-1,j]$; $B[i,j] \leftarrow \text{"\textasciitilde{~}"}$;

10. Else $C[i,j] \leftarrow C[i,j-1]$; $B[i,j] \leftarrow \text{"\textasciitilde{~}"}$;

11. Return C and B

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解 

第四步：获取构造最优值的信息

- 基本思想

- 从 $B[m, n]$ 开始按指针搜索
- 若 $B[i, j] = “\nwarrow”$ ，则 $x_i = y_j$ 是LCS的一个元素
- 如此找到的“LCS”是 X 与 Y 的LCS

构造最优解

Print-LCS(B,X,i,j)

1. IF $i=0$ or $j=0$ THEN Return;

2. IF $B[i, j] = "\diagup"$

3. THEN Print-LCS(B, X, $i-1, j-1$), Print x_i

4. ELSE IF $B[i, j] = "\uparrow"$

5. THEN Print-LCS(B, X, $i-1, j$)

6. ELSE Print-LCS(B, X, $i, j-1$)

Print – LCS(B, X, length(X), length(Y)) 可打印出 *X* 与 *Y* 的 *LCS*

算法复杂性

- 时间复杂性
 - 计算代价的时间
 - (i, j) 两层循环, i 循环 m 步, j 循环 n 步
 - $O(mn)$
 - 初始化的时间: $O(m + n)$
 - 总时间复杂性为: $O(mn)$
- 空间复杂性
 - 使用数组 C 和 B
 - 需要空间 $O(mn)$

示例

j	0	1	2	3	4	5	6	7	8	9	10
i	y_j	a	m	p	u	t	a	t	i	o	n
0	x_i	0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0	0↑	0↑	1↘	1←	1←	1←	1←	1←	1←
3	a	0	1↘	1←	1↑	1↑	1↑	2↘	2←	2←	2←
4	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	3↘
5	k	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	3↑
6	i	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↘	3←
7	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↑	4↘
8	g	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↑	4↑

示例

j	0	1	2	3	4	5	6	7	8	9	10
i	y_j	a	m	p	u	t	a	t	i	o	n
0	x_i	0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0	0↑	0↑	1↖	1←	1←	1←	1←	1←	1←
3	a	0	1↖	1←	1↑	1↑	1↑	2↖	2←	2←	2←
4	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	3↖
5	k	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	3↑
6	i	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↖	3↑
7	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↑	3↑
8	g	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↑	3↑

最长公共子序列是 **pain**

示例 2

$X = \langle A, B, C, B, D, A, B \rangle$; $Y = \langle B, D, C, A, B, A \rangle$

	A	B	C	B	D	A	B
	0	0	0	0	0	0	0
B	0	0	1	1	1	1	1
D	0	0	1	1	1	2	2
C	0	0	1	2	2	2	2
A	0	1	1	2	2	3	3
B	0	1	2	2	3	3	4
A	0	1	2	2	3	3	4

本讲内容

7.1 动态规划的原理

7.2 矩阵乘法问题

7.3 最长公共子序列

7.4 0-1背包问题

7.5 最优二叉搜索树

背包问题定义

给定 n 种物品和一个背包，物品 i 的重量是 w_i ，价值 v_i ，背包容量为 C ，问如何选择装入背包的物品，使装入背包中的物品的总价值最大？

根据是否对每类物品的数量进行限定，可分为两类问题：

- 完全背包问题（Unbounded Knapsack）：

假设所有物品我们都有无限份

- 0-1背包问题（0-1 Knapsack）：

假设所有物品，我们都至多一份，即每种物品只能选择完全装入或不装入，一个物品至多装入一次。



Capacity: 10

Item:

Weight:

Value:



6

20



2

8



4

14



3

13



11

35

• 完全背包问题 (Unbounded Knapsack) :

- 假设所有物品，我们都有无限份
- 问如何选择装入背包的物品，使装入背包中的物品的总价值最大？



Total weight: 10

Total value: 42

➡ • 0-1背包问题 (0-1 Knapsack) :

- 假设所有物品，我们都只有一份
- 问如何选择装入背包的物品，使装入背包中的物品的总价值最大？



Total weight: 9

Total value: 35

符号定义

Item:



Weight:

 w_1 w_2 w_3

...

 w_n

Value:

 v_1 v_2 v_3

...

 v_n 

Capacity: C

完全背包问题

- 输入: $C > 0, w_i > 0, v_i > 0, 1 \leq i \leq n$
- 输出: $(h_1, h_2, \dots, h_n)$, 满足

$$\sum_{1 \leq i \leq n} w_i h_i \leq C, \sum_{1 \leq i \leq n} v_i h_i \text{ 最大}$$

等价的整数规划问题

$$\begin{aligned} & \max_h \sum_{1 \leq i \leq n} v_i h_i \\ & s. t. \sum_{1 \leq i \leq n} w_i h_i \leq C \end{aligned}$$

0-1背包问题

- 输入: $C > 0, w_i > 0, v_i > 0, 1 \leq i \leq n$
- 输出: $(h_1, h_2, \dots, h_n), h_i \in \{0, 1\}$, 满足

$$\sum_{1 \leq i \leq n} w_i h_i \leq C, \sum_{1 \leq i \leq n} v_i h_i \text{ 最大}$$

等价的0-1整数规划问题

$$\begin{aligned} & \max_h \sum_{1 \leq i \leq n} v_i h_i \\ \text{s. t. } & \begin{cases} \sum_{1 \leq i \leq n} w_i h_i \leq C \\ h_i \in \{0, 1\} \quad (1 \leq i \leq n) \end{cases} \end{aligned}$$

利用暴力搜索方法求解0-1背包问题, 每件物品都存在装入和不装入两种情况, 所以总的时间复杂度是 $O(2^n)$ 。

使用动态规划

- 分析优化解的结构 
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

优化子结构分析: try 1

- 子问题:
 - 考虑更小的背包



首先解小背包



然后，解大背包



然后，解更大的背包

优化子结构分析: try 1

- 这种方式失效

- 每类物品只有一份
- 求解子问题时，我们需要知道哪些物品已经被其子问题用过，哪些没有被用过



乌龟我已经不能再被用了

优化子结构分析: try 2

- 子问题:
 - 有更少物品种类的0-1 背包问题

逐渐增加背包的承重

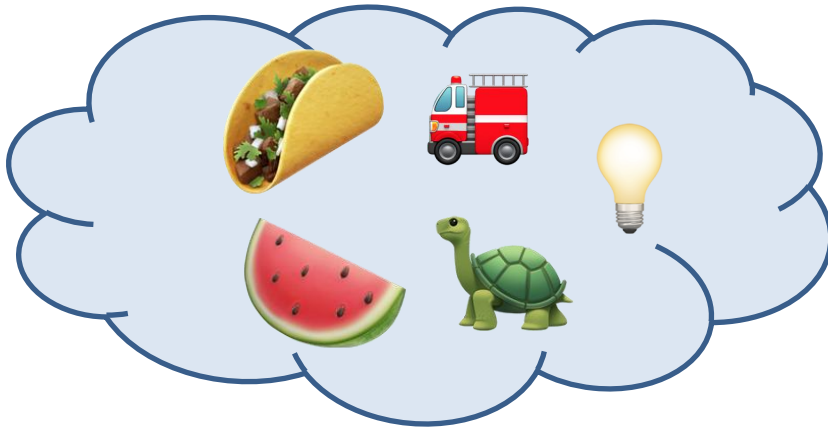
逐渐增加可选物品的种类



因此，0-1背包问题，需要二维矩阵数据结构

优化子结构分析: try 2

- 二维矩阵数据结构包括 x 和 j 两个维度



j : 前 j 个物品



x : 承重为 x 的背包

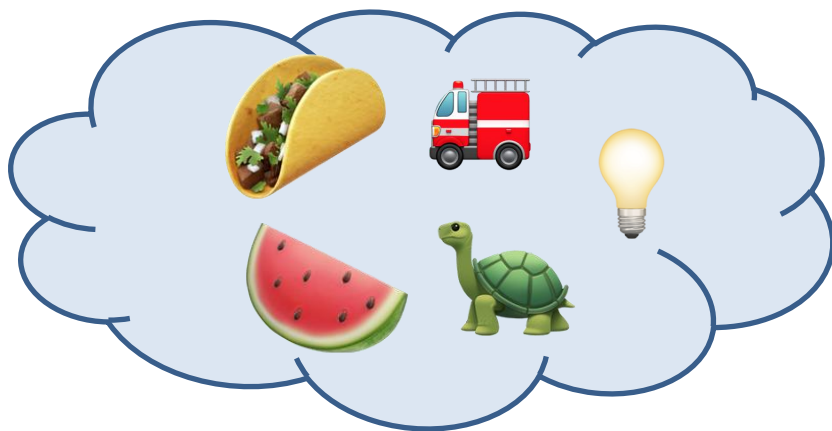
定义 $K[x, j]$ 为如下0-1背包问题的最优值

两种情况



item j

- **Case 1:** 前 j 个物品的子问题的最优解, 不包含第 j 个物品
- **Case 2:** 前 j 个物品的子问题的最优解, 包含了第 j 个物品



前 j 个物品



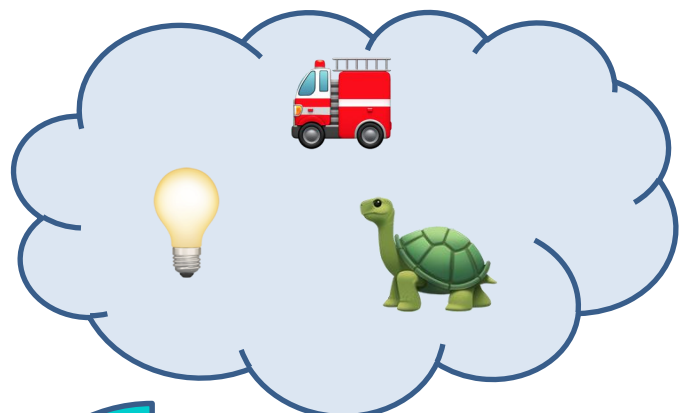
承重为 x 的背包

两种情况



item j

- **Case 1:** 前 j 个物品的子问题的最优解，不包含第 j 个物品



前 j 个物品



背包承重为 x ，前 j 个物品的
0-1背包问题，最大价值为 K

- 可以得到背包承重为 x ，考虑前 $j - 1$ 个物品的0-1背包问题



前 $j - 1$ 个物品



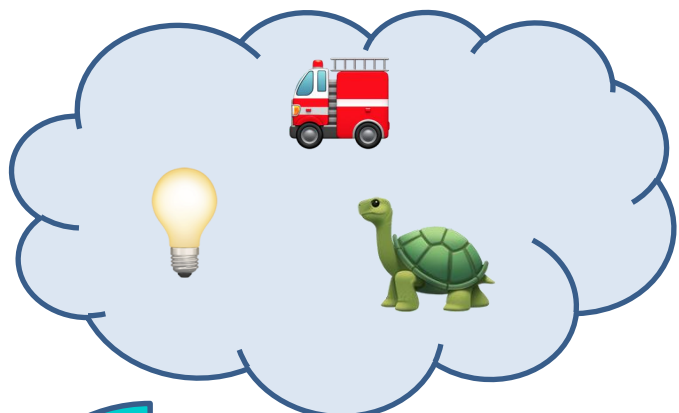
背包承重为 x ，前 $j - 1$ 个物品
的0-1背包问题，最大价值为
 $K[x, j - 1]$

两种情况



item j

- **Case 2:** 前 j 个物品的子问题的最优解，包含第 j 个物品



前 j 个物品



背包承重为 x ，前 j 个物品的
0-1背包问题，最大价值为 K

- 可以得到背包承重为 $x - w_j$ ，考虑前 $j - 1$ 个物品的0-1背包问题



前 $j - 1$ 个物品



背包承重为 $x - w_j$ ，前 $j - 1$
个物品的0-1背包问题，最大
价值为 $K[x - w_j, j - 1] + v_j$

第一步：分析优化解的结构

定理（优化子结构）： 如果 $(y_1, y_2, \dots, y_n)$ 是 0-1 背包问题的优化解，则 $(y_1, \dots, y_{n-1})$ 是如下子问题的优化解：

$$\begin{aligned} & \max_h \sum_{1 \leq i \leq n-1} v_i h_i \\ \text{s.t.} & \begin{cases} \sum_{1 \leq i \leq n-1} w_i h_i \leq C - w_n y_n \\ h_i \in \{0, 1\} \quad (1 \leq i \leq n-1) \end{cases} \end{aligned}$$

证明（反证法）：

如果 $(y_1, \dots, y_{n-1})$ 不是以上子问题优化解，则存在 $(z_1, \dots, z_{n-1})$ 是此子问题更优的解。于是， $(z_1, \dots, z_{n-1}, y_n)$ 是原问题比 $(y_1, y_2, \dots, y_n)$ 更优的解，矛盾。

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价 
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

第二步：递归地定义最优值的代价

- 定义 $K[x, j]$ 为如下0-1背包问题的最优值：

- 背包承重为 x ,
- 考虑前 j 个物品。

$$K[x, j] = \max\{K[x, j - 1], K[x - w_j, j - 1] + v_j\}$$

Case 1

Case 2

- 初始值
 - $K[x, 0] = 0$ 可选物品为空
 - $K[0, j] = 0$ 背包承重为0

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解



第三步：自底向上地计算优化解的代价

Zero-One-Knapsack(W , weights, values)

Input: 背包承重 C ，每类物品对应的重量和价值数组weights, values

Output: 背包所能装的最大价值以及对应的所装物品

Zero-One-Knapsack(W , weights, values)

过程: Zero-One-Knapsack(W , weights, values)

```
1.  $n \leftarrow \text{length}(\text{weights})$ 
2. For  $x \leftarrow 0$  To  $C$  Do
3.    $K[x, 0] \leftarrow 0$  /*初始化第0列*/
4. For  $j \leftarrow 1$  To  $n$  Do
5.    $K[0, j] \leftarrow 0$  /*初始化第0行*/
6. For  $x \leftarrow 0$  To  $C$  Do
7.   For  $j \leftarrow 1$  To  $n$  Do
8.      $K[x, j] \leftarrow K[x, j-1]$  Case 1
9.     IF  $w_j \leq x$  THEN
10.       $K[x, j] \leftarrow \max\{K[x, j], K[x-w_j, j-1] + v_j\}$  Case 2
11. return  $K[C, n]$ 
```

时间复杂度 $O(nC)$

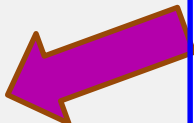
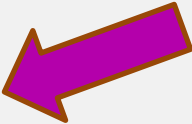
使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解 

第四步：获取构造最优值的信息

过程：Zero-One-Knapsack($C, \text{weights}, \text{values}$)

```
1.  $n \leftarrow \text{length}(\text{weights})$ 
2. For  $x \leftarrow 0$  To  $C$  Do
3.   For  $j \leftarrow 1$  To  $n$  Do
4.     ITEMS[ $x, j$ ]  $\leftarrow \emptyset$ 
5. For  $x \leftarrow 0$  To  $C$  Do
6.    $K[x, 0] \leftarrow 0$  /*初始化第0列*/
7. For  $j \leftarrow 1$  To  $n$  Do
8.    $K[0, j] \leftarrow 0$  /*初始化第0行*/
9. For  $x \leftarrow 0$  To  $C$  Do
10.  For  $j \leftarrow 1$  To  $n$  Do
11.     $K[x, j] \leftarrow K[x, j-1]$ 
12.    IF  $w_j \leq x$  THEN
13.       $K[x, j] \leftarrow \max\{ K[x, j], K[x - w_j, j-1] + v_j \}$ 
14.      IF  $K[x, j]$  was updated THEN
15.        ITEMS[ $x, j$ ]  $\leftarrow \text{ITEMS}[x - w_j, j-1] \cup \{\text{ITEM } j\}$ 
16.      ELSE
17.        ITEMS[ $x, j$ ]  $\leftarrow \text{ITEMS}[x, j-1]$ 
18. Return  $K[C, n], \text{ITEMS}[C, n]$ 
```

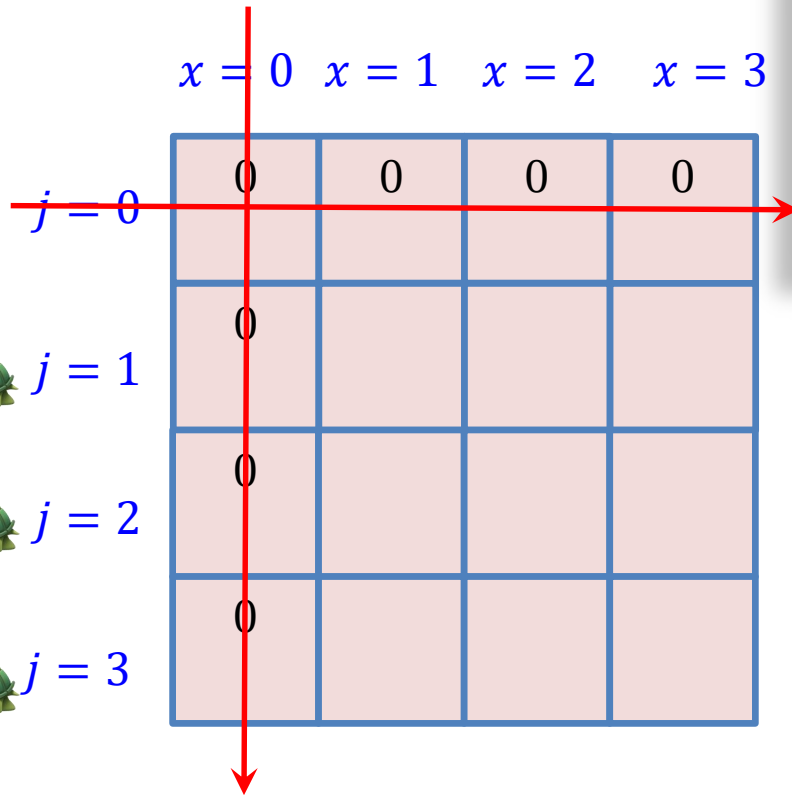


示例

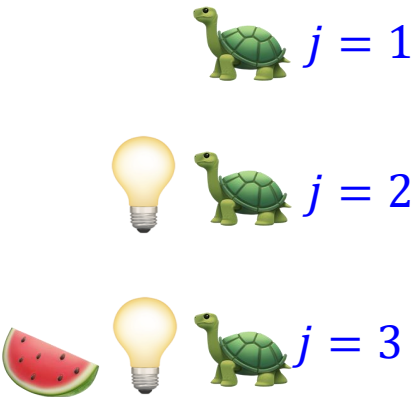
```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF wj ≤ x THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]

```



$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF wj ≤ x THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]

```

$x = 0$ $x = 1$ $x = 2$ $x = 3$

$j = 0$

	0	0	0	0
	0	0		
	0			
	0			

$j = 1$

$j = 2$

$j = 3$

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

$j = 1$

$j = 2$

$j = 3$

	0	0	0	0
	0	1		
	0			
	0			

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x -  $w_j$ ,j-1] +  $v_j$  }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x -  $w_j$ ,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]

```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1		
---	---	--	--

$j = 2$

0	1		
---	---	--	--

$j = 3$

0			
---	--	--	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1		
---	---	--	--

$j = 2$

0	1		
---	---	--	--

$j = 3$

0	1		
---	---	--	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]

```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	0	
---	---	---	--

$j = 2$

0	1		
---	---	--	--

$j = 3$

0	1		
---	---	--	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	
---	---	---	--

$j = 2$

0	1		
---	---	--	--

$j = 3$

0	1		
---	---	--	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]
    
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	
---	---	---	--

$j = 2$

0	1	1	
---	---	---	--

$j = 3$

0	1		
---	---	--	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]

```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

	0	0	0	0
--	---	---	---	---

$j = 1$



0	1	1	
---	---	---	--



$j = 2$



0	1	4	
---	---	---	--



$j = 3$



0	1		
---	---	--	--



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF wj ≤ x THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF k[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]
    
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	
---	---	---	--

$j = 2$

0	1	4	
---	---	---	--

$j = 3$

0	1	4	
---	---	---	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]
    
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	0
---	---	---	---

$j = 2$

0	1	4	
---	---	---	--

$j = 3$

0	1	4	
---	---	---	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]
    
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	1
---	---	---	---

$j = 2$

0	1	4	
---	---	---	--

$j = 3$

0	1	4	
---	---	---	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]
    
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	1
---	---	---	---

$j = 2$

0	1	4	1
---	---	---	---

$j = 3$

0	1	4	
---	---	---	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]

```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	1
---	---	---	---

$j = 2$

0	1	4	5
---	---	---	---

$j = 3$

0	1	4	
---	---	---	--

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]
    
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	1
---	---	---	---

$j = 2$

0	1	4	5
---	---	---	---

$j = 3$

0	1	4	5
---	---	---	---

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]
    
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	1
---	---	---	---

$j = 2$

0	1	4	5
---	---	---	---

$j = 3$

0	1	4	6
---	---	---	---

```

1. n ← length(weights)
2. For x ← 0 To C Do
3.   For j ← 1 To n Do
4.     ITEMS[x,j] ← ∅
5.   For x ← 0 To C Do
6.     K[x,0] ← 0 /*初始化第0列*/
7.   For j ← 1 To n Do
8.     K[0,j] ← 0 /*初始化第0行*/
9.   For x ← 0 To C Do
10.    For j ← 1 To n Do
11.      K[x,j] ← K[x,j-1]
12.      IF  $w_j \leq x$  THEN
13.        K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.        IF K[x,j] was updated THEN
15.          ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.      ELSE
17.        ITEMS[x,j] ← ITEMS[x,j-1]
18. Return K[C,n], ITEMS[C,n]
    
```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

示例

$x = 0 \quad x = 1 \quad x = 2 \quad x = 3$

$j = 0$

0	0	0	0
---	---	---	---

$j = 1$

0	1	1	1
---	---	---	---

$j = 2$

0	1	4	5
---	---	---	---

$j = 3$

0	1	4	6
---	---	---	---

```

1.  n ← length(weights)
2.  For x ← 0 To C Do
3.    For j ← 1 To n Do
4.      ITEMS[x,j] ← ∅
5.  For x ← 0 To C Do
6.    K[x,0] ← 0 /*初始化第0列*/
7.  For j ← 1 To n Do
8.    K[0,j] ← 0 /*初始化第0行*/
9.  For x ← 0 To C Do
10.   For j ← 1 To n Do
11.     K[x,j] ← K[x,j-1]
12.     IF  $w_j \leq x$  THEN
13.       K[x,j] ← max{ K[x,j], K[x - wj,j-1] + vj }
14.       IF K[x,j] was updated THEN
15.         ITEMS[x,j] ← ITEMS[x - wj,j-1] ∪ {ITEM j}
16.     ELSE
17.       ITEMS[x,j] ← ITEMS[x,j-1]
18.  Return K[C,n], ITEMS[C,n]

```

$$K[x,j] = \max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$



current
entry



relevant
previous entry

Item:

Weight:

Value:



1

1



2

4



3

6



Capacity: 3

本讲内容

7.1 动态规划的原理

7.2 矩阵乘法问题

7.3 最长公共子序列

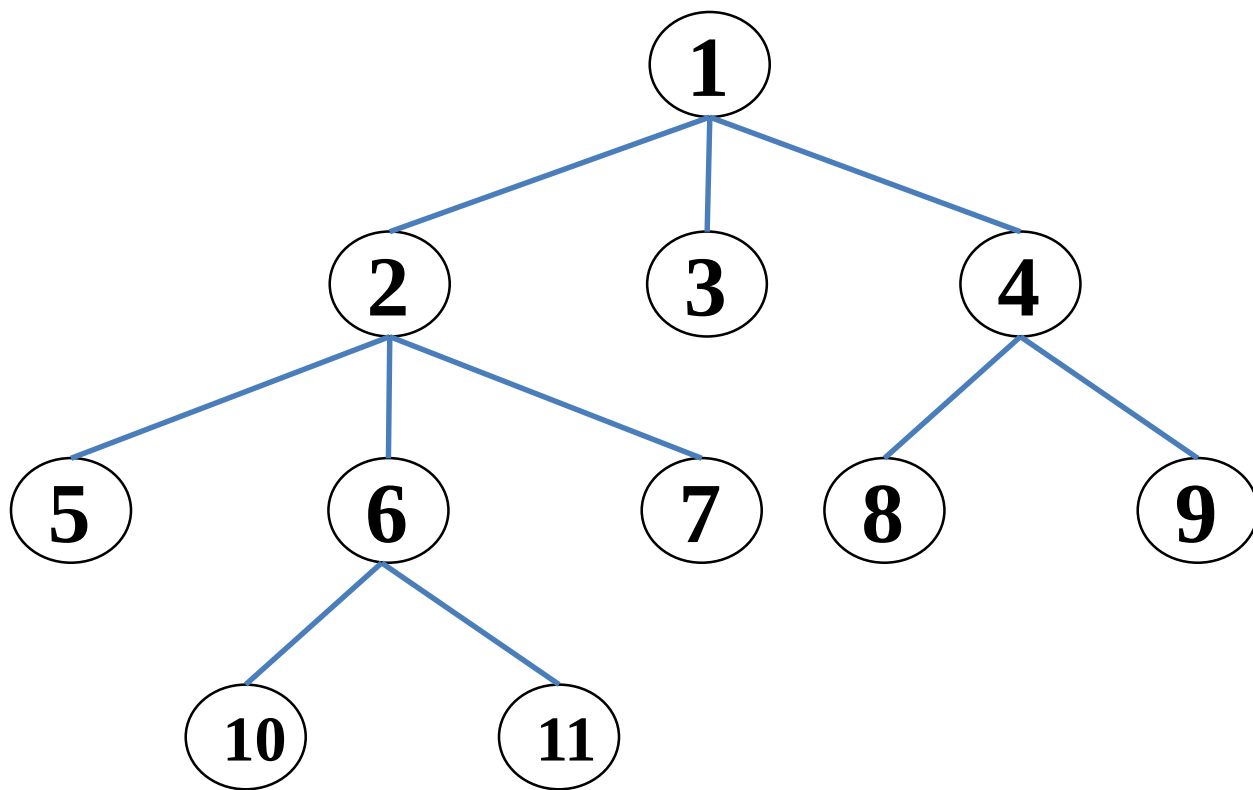
7.4 0-1背包问题

7.5 最优二叉搜索树

树

树形结构是一种重要的非线性结构。树是 n 个结点的有限集合，在任一棵非空树中：

- (1) 有且仅有一个称为根的结点。
- (2) 其余结点可分为 m 个互不相交的集合，而且这些集合中的每一集合都本身又是一棵树，称为根的子树。

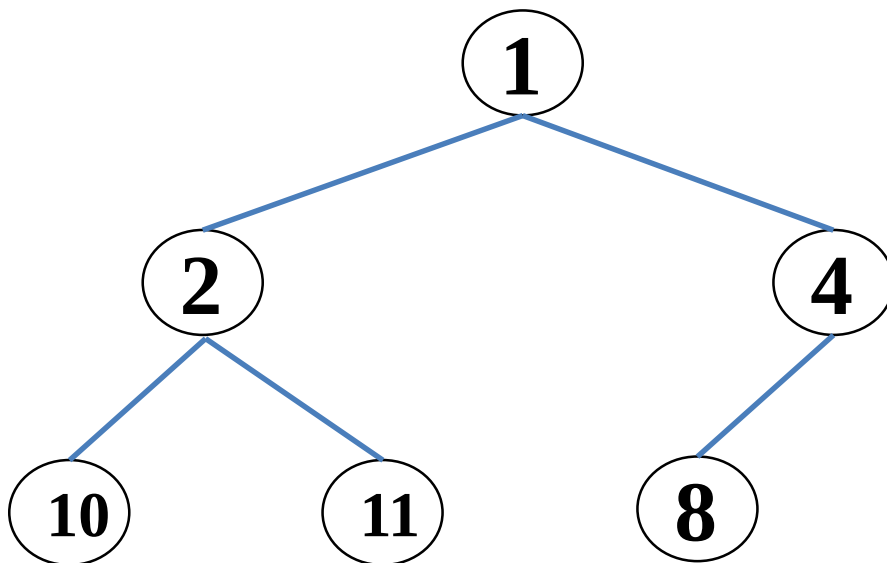


二叉树

概念：二叉树或为空树，或由根及两颗不相交的左、右子树构成，并且左、右子树本身也是二叉树。

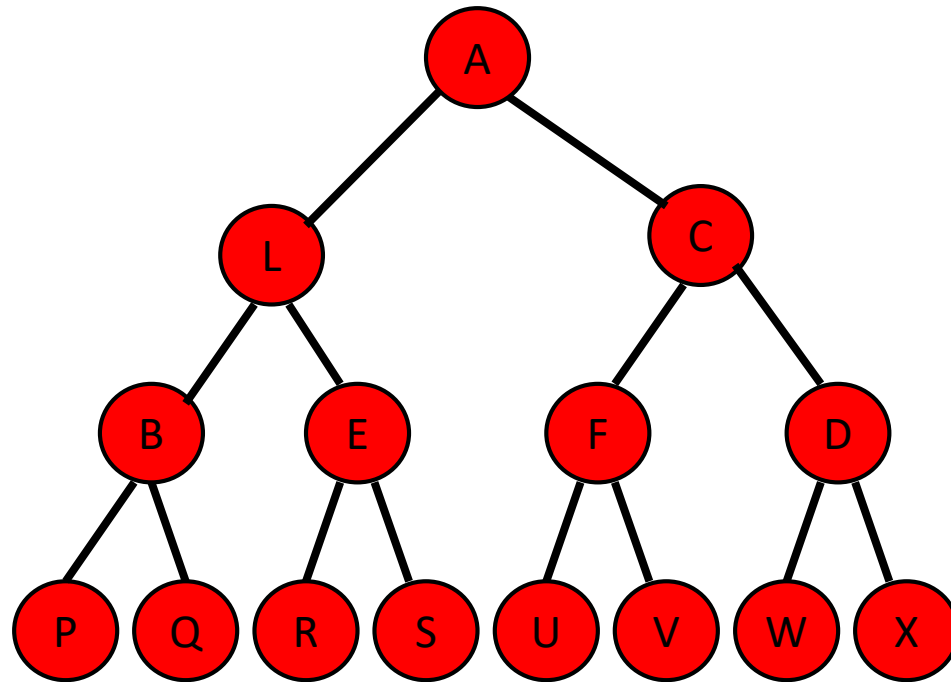
特点：

- 二叉树中每个结点最多有两棵子树；即二叉树每个结点度小于等于2；
- 左、右子树不能颠倒——有序树；
- 二叉树是递归结构，在二叉树的定义中又用到了二叉树的概念；



满二叉树

特殊二叉树，每层都是满的

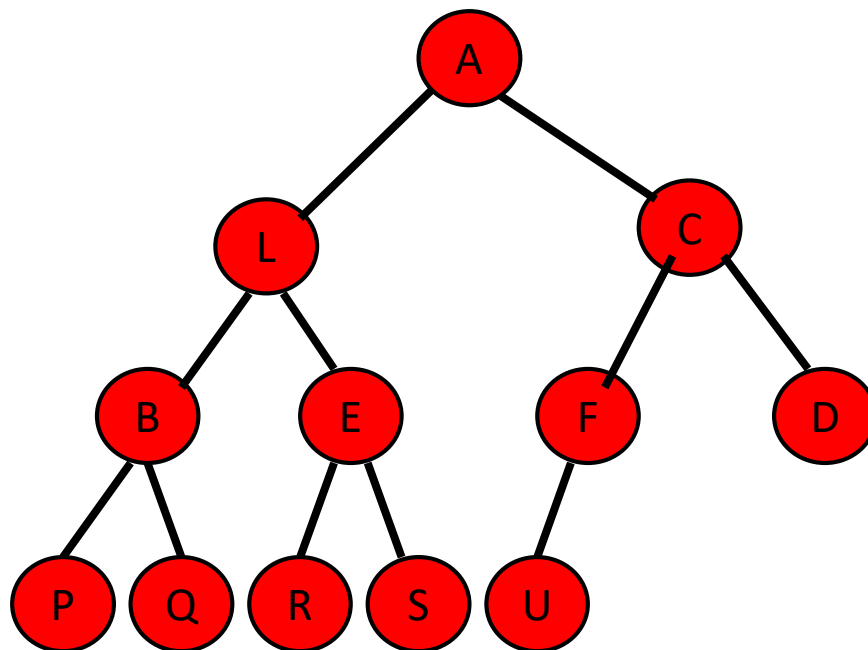


完全二叉树

特殊二叉树，除了最后一层，每一层都是满的

满二叉树

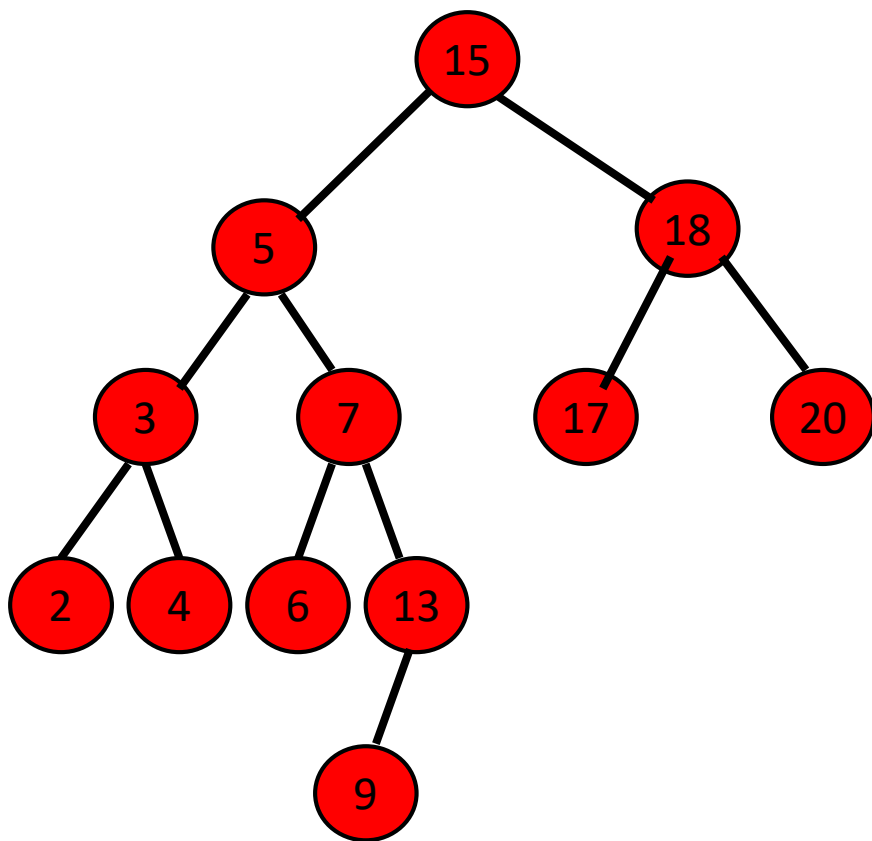
从满二叉树最底层从右向左依次去掉若干结点形成的



二叉搜索树

特殊二叉树：

- 左子树不空，则左子树所有结点值均小于该子树父结点值
- 右子树不空，则右子树所有结点值均大于该子树父结点值
- 左右子树都是二叉搜索树
- 中序遍历二叉查找树就可以得到一个有序序列



最优二叉搜索树 问题的定义

- 二叉搜索树 T

- 结点

- $K = \{k_1, k_2, \dots, k_n\}$: 已排序的
关键字序列

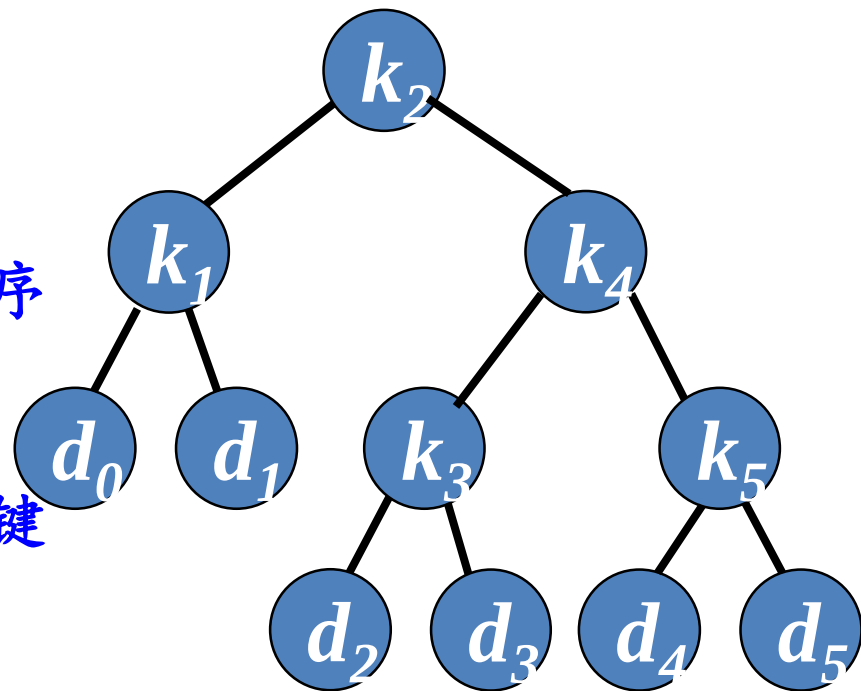
- $D = \{d_0, d_1, \dots, d_n\}$: 伪关键字序列

- d_i 对应区间 (k_i, k_{i+1})

- d_0 对应区间 $(-\infty, k_1)$

- d_n 对应区间 $(k_n, +\infty)$

特点：对于树中结点 X ，左子树中所有关键字值小于 X ，右子树中所有关键字值大于 X 。

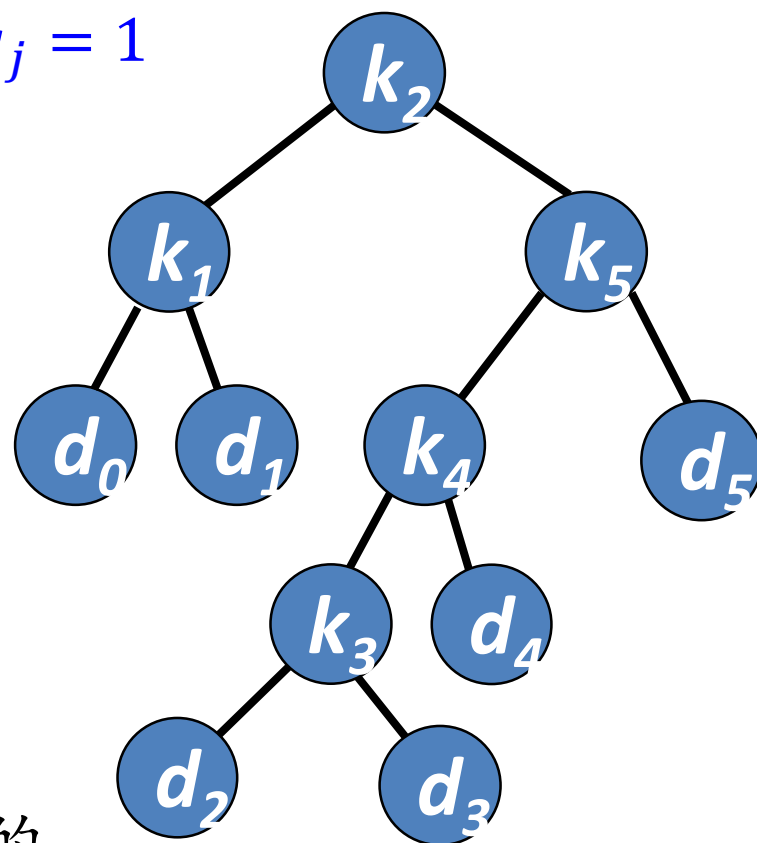
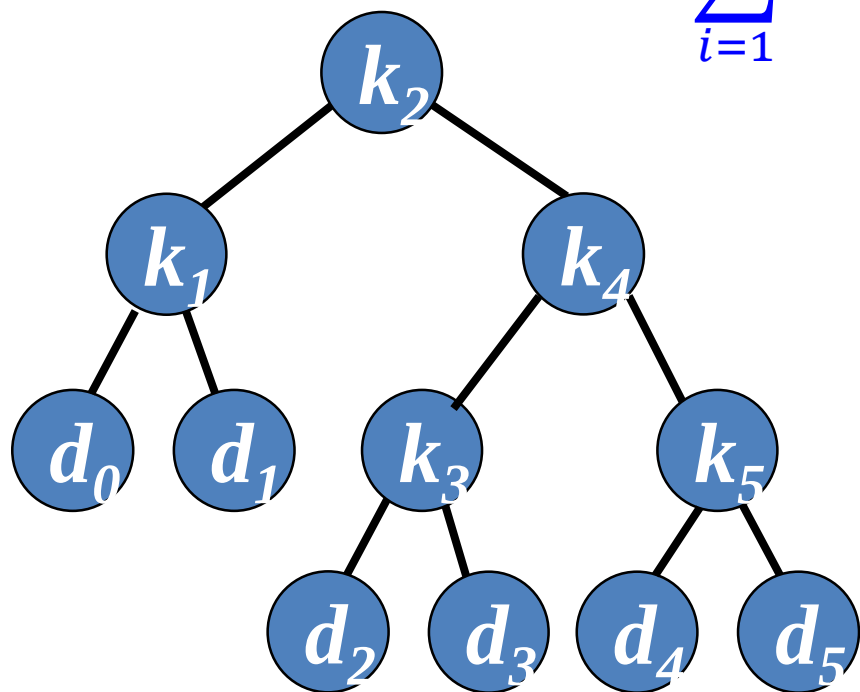


最优二叉搜索树 问题的定义

- 搜索 k_i 的概率为 p_i
- 搜索 d_i 的概率为 q_i

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$\sum_{i=1}^n p_i + \sum_{j=0}^n q_j = 1$$



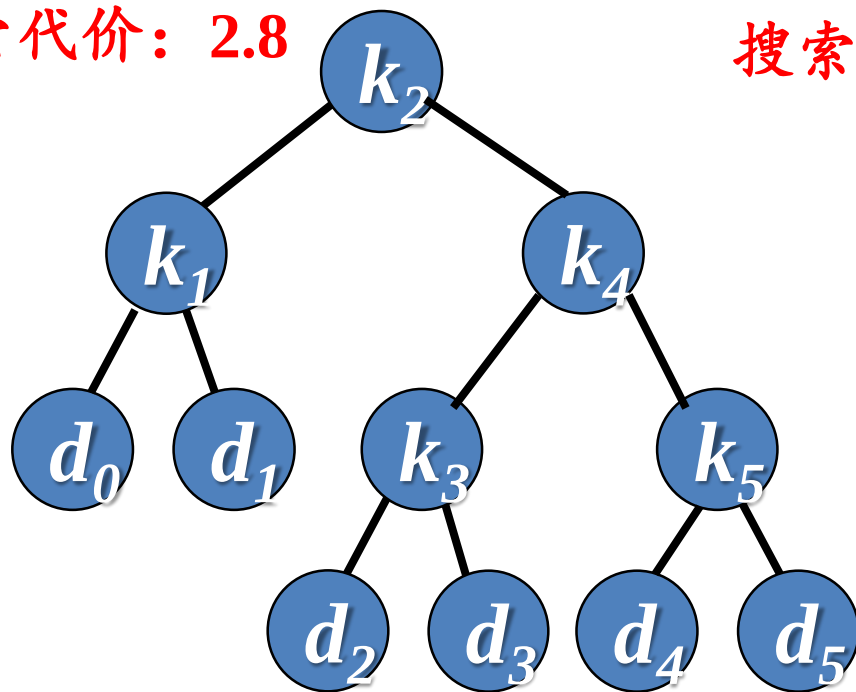
每个关键字的搜索代价 **等于** 需要访问的
 结点个数 **等于** 关键字所在结点深度+1

最优二叉搜索树 问题的定义

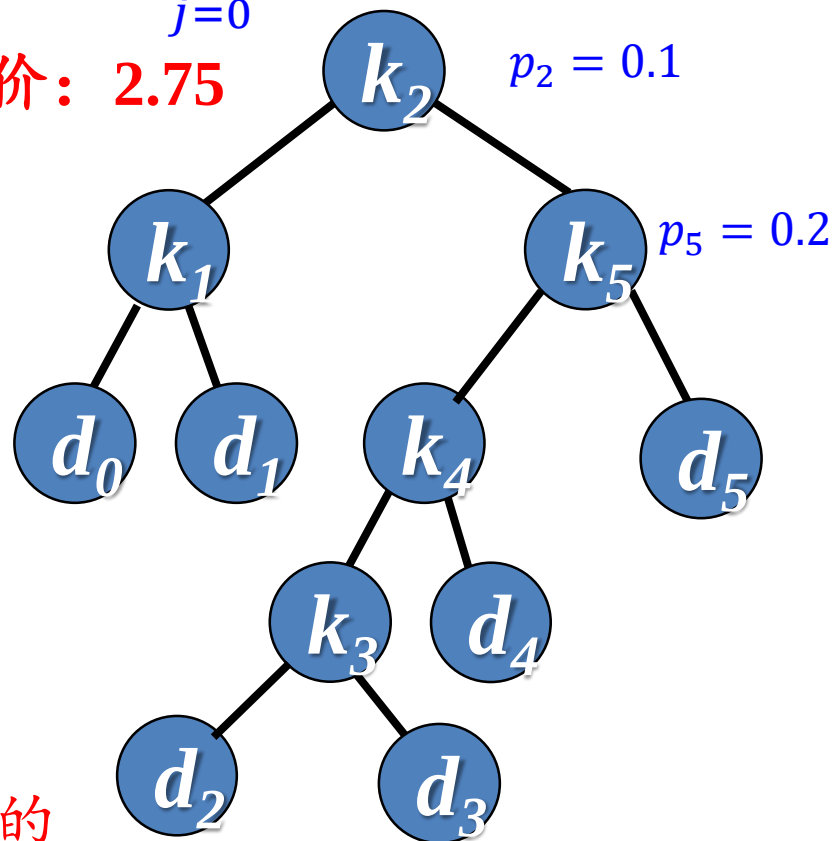
- 搜索树的期望代价

$$E(T) = \sum_{i=1}^n (\text{depth}_T(k_i) + 1)p_i + \sum_{j=0}^n (\text{depth}_T(d_j) + 1)q_j$$

搜索代价: 2.8



搜索代价: 2.75



- 最优二叉搜索树不一定是高度最矮的
- 概率高的关键字不一定出现在二叉搜索树的根结点

最优二叉搜索树 问题的定义

输入：

$K = \{k_1, k_2, \dots, k_n\}$, $k_1 < k_2 < \dots < k_n$, (有序的序列)

$D = \{d_0, d_1, \dots, d_n\}$

$P = \{p_1, p_2, \dots, p_n\}$, p_i 为搜索关键字 k_i 的概率

$Q = \{q_0, q_1, \dots, q_n\}$, q_i 为搜索伪关键字 d_i 的概率

输出：

K 的最优二叉搜索树 T , $E(T)$ 最小

使用动态规划

- 分析优化解的结构 
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

优化子结构的分析

定理（优化子结构）：

如果最优二叉搜索树 T 具有包含关键字集合 $\{k_i, k_{i+1}, \dots, k_j\}$ 子树 T' ，则 T' 必是关于关键字集合 $\{k_i, k_{i+1}, \dots, k_j\}$ 子问题的优化解。

证明（反证法）：

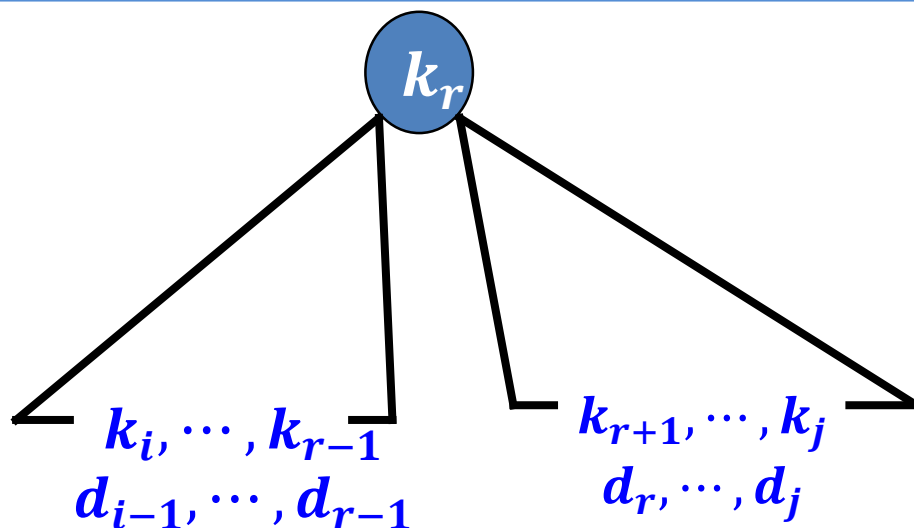
若不然，必有关关键字集 $\{k_i, k_{i+1}, \dots, k_j\}$ 子树 T'' ， T'' 的期望搜索代价低于 T' ；

用 T'' 替换 T 中的 T' ，可以得到一个期望搜索代价比 T 小的原始问题的二叉搜索树，与 T 是最优解矛盾。

为什么是 $\{k_i, k_{i+1}, \dots, k_j\}$ ？

优化子结构的分析

- 用优化子结构从子问题的优化解构造原问题的优化解
 - $K = \{k_i, k_{i+1}, \dots, k_j\}$ 的优化解的根结点为 K 中某个 k_r ;
 - 对于 $k_r \in K$, 确定 $\{k_i, \dots, k_{r-1}\}$ 和 $\{k_{r+1}, \dots, k_j\}$ 的优化解, 我们就可以求出 K 的优化解。



空子树:

如果 $r = i$, 左子树 $\{k_i, \dots, k_{r-1}\}$ 不包含任何关键字, 仅包含伪关键字 d_{i-1}

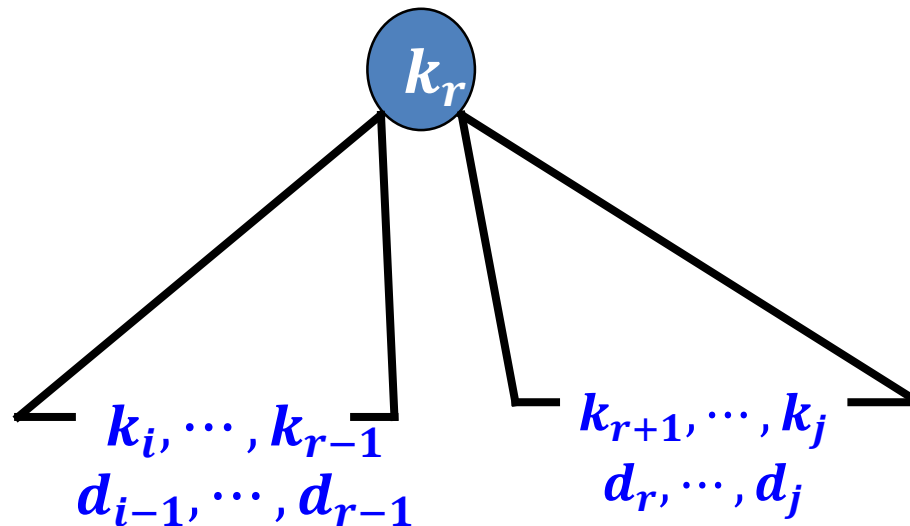
如果 $r = j$, 右子树 $\{k_{r+1}, \dots, k_j\}$ 不包含任何关键字, 仅包含伪关键字 d_j

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价 
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

建立优化解的搜索代价递归方程

- 令 $E(i, j)$ 为 $\{k_i, \dots, k_j\}$ 和 $\{d_{i-1}, d_i, \dots, d_j\}$ 的优化解 T_{ij} 的期望搜索代价, $W(i, j)$ 为 $\{k_i, \dots, k_j\}$ 和 $\{d_{i-1}, \dots, d_j\}$ 的概率之和
 - 当 $j = i - 1$ 时, T_{ij} 中只有叶结点 d_{i-1} , $E(i, i - 1) = q_{i-1}$
 - 当 $j \geq i$ 时, 选择一个 $K_r \in \{k_i, \dots, k_j\}$: $= q_j$



根结点的搜索代价

$$E(i, j) = P_r + (E(i, r - 1) + W(i, r - 1)) + (E(r + 1, j) + W(r + 1, j))$$

左子树

右子树

建立优化解的搜索代价递归方程

- 当把左右优化子树放进 T_{ij} 时，每个结点的深度增加1

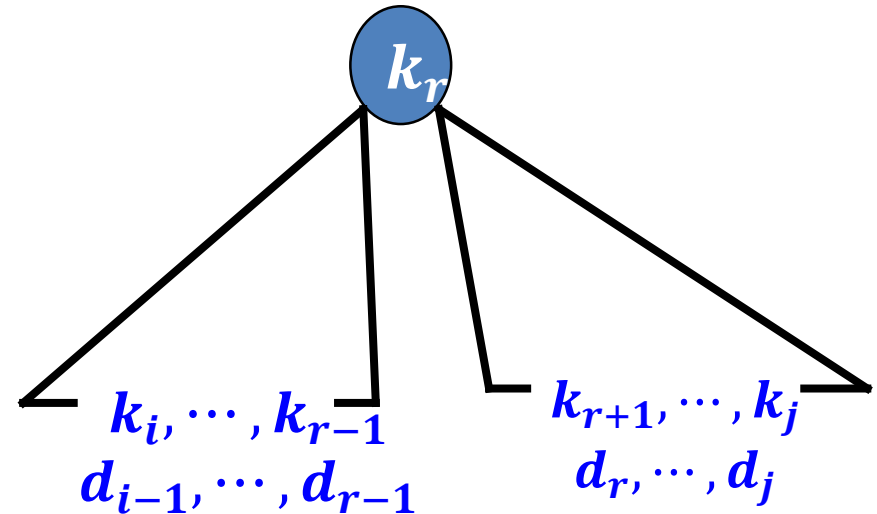
$$1、W(i, r-1) = \sum_{l=i}^{r-1} P_l + \sum_{l=i-1}^{r-1} q_l$$

$$2、W(r+1, j) = \sum_{l=r+1}^j P_l + \sum_{l=r}^j q_l$$

$$3、W(i, j) = W(i, r-1) + W(r+1, j) + P_r$$

$$= \sum_{l=i}^{r-1} P_l + \sum_{l=r+1}^j P_l + \sum_{l=i-1}^{r-1} q_l + \sum_{l=r}^j q_l + P_r$$

$$= \sum_{l=i}^j P_l + \sum_{l=i-1}^j q_l$$



因此， $W(i, j)$ 与 r 没有关系，无论在哪里分， $W(i, j)$ 都是相同的！

$$E(i, j) = E(i, r-1) + E(r+1, j) + W(i, j)$$

建立优化解的搜索代价递归方程

$$E(i, j) = \begin{cases} q_{i-1} = q_j & \text{if } j = i - 1 \\ \min_{i \leq r \leq j} \{E(i, r - 1) + E(r + 1, j) + W(i, j)\} & \text{if } j \geq i \end{cases}$$

$$W(i, j) = \sum_{l=i}^j P_l + \sum_{l=i-1}^j q_l$$

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解



自下而上计算优化解的搜索代价

$$q_0 = E(1, 0) \quad E(1, 1) \quad E(1, 2) \quad E(1, 3) \quad E(1, 4)$$

$$q_1 = E(2, 1) \quad E(2, 2) \quad E(2, 3) \quad E(2, 4)$$

$$q_2 = E(3, 2) \quad E(3, 3) \quad E(3, 4)$$

$$q_3 = E(4, 3) \quad E(4, 4)$$

$$q_4 = E(5, 4)$$

矩阵 E 有什么特点?

行号 i 从1到 $n+1$

列号 j 从0到 n (n 是关键字的个数)

$$E(i, j) = \begin{cases} q_{i-1} = q_j & \text{if } j = i - 1 \\ \min_{i \leq r \leq j} \{E(i, r-1) + E(r+1, j) + W(i, j)\} & \text{if } j \geq i \end{cases}$$

自下而上计算优化解的搜索代价

$W(i, j)$ 与 r 没有关系, 无论在哪里分, $W(i, j)$ 都是相同的! 设定从 j 处分

$$W(i, i-1) = q_{i-1}; \quad W(j+1, j) = q_j$$

$$\begin{aligned} W(i, j) &= W(i, j-1) + W(j+1, j) + p_j \\ &= W(i, j-1) + p_j + q_j \end{aligned}$$

$q_0 = W(1, 0) \ W(1, 1) \ W(1, 2) \ W(1, 3) \ W(1, 4)$

$q_1 = W(2, 1) \ W(2, 2) \ W(2, 3) \ W(2, 4)$

$q_2 = W(3, 2) \ W(3, 3) \ W(3, 4)$

$q_3 = W(4, 3) \ W(4, 4)$

$q_4 = W(5, 4)$

自下而上计算优化解的搜索代价

- 数据结构

- $E[1:n+1; 0:n]$: 存储优化解搜索代价
- $W[1:n+1; 0:n]$: 存储代价增量
- $Root[1:n; 1:n]$: $root(i, j)$ 记录子问题 $\{k_i, \dots, k_j\}$ 优化解的根结点

自下而上计算优化解的搜索代价

Optimal-BST

Input: K和D的概率数组p、q

Output: 搜索代价E, 根结点Root

执行: Optimal-BST(p, q)

过程: Optimal-BST(p, q)

1. $n \leftarrow \text{length}(p)$

2. **For** $i \leftarrow 1$ **To** $n+1$ **Do**

3. $E(i, i-1) \leftarrow q_{i-1};$

4. $W(i, i-1) \leftarrow q_{i-1};$

5. **For** $l \leftarrow 1$ **To** n **Do**

6. **For** $i \leftarrow 1$ **To** $n-l+1$ **Do**

7. $j \leftarrow i+l-1;$

8. $E(i, j) \leftarrow \infty;$

9. $W(i, j) \leftarrow W(i, j-1) + p_j + q_j;$

10. **For** $r=i$ **To** j **Do**

11. $t \leftarrow E(i, r-1) + E(r+1, j) + W(i, j);$

12. **IF** $t < E(i, j)$ **Then**

13. $E(i, j) \leftarrow t; \text{Root}(i, j) \leftarrow r;$

14. **Return** E and Root

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解 

思考：优化解的构造算法

示例

$$E(i, j) = \begin{cases} q_{i-1} = q_j & \text{if } j = i - 1 \\ \min_{i \leq r \leq j} \{E(i, r-1) + E(r+1, j) + W(i, j)\} & \text{if } j \geq i \end{cases}$$

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

0	1	2	3	4	5	
0.05	0.45	0.90	1.25	1.75	2.75	1
	0.10	0.40	0.70	1.20	2.00	2
		0.05	0.25	0.60	1.30	3
			0.05	0.30	0.90	4
				0.05	0.50	5
					0.10	6

(a) $E(i, j)$

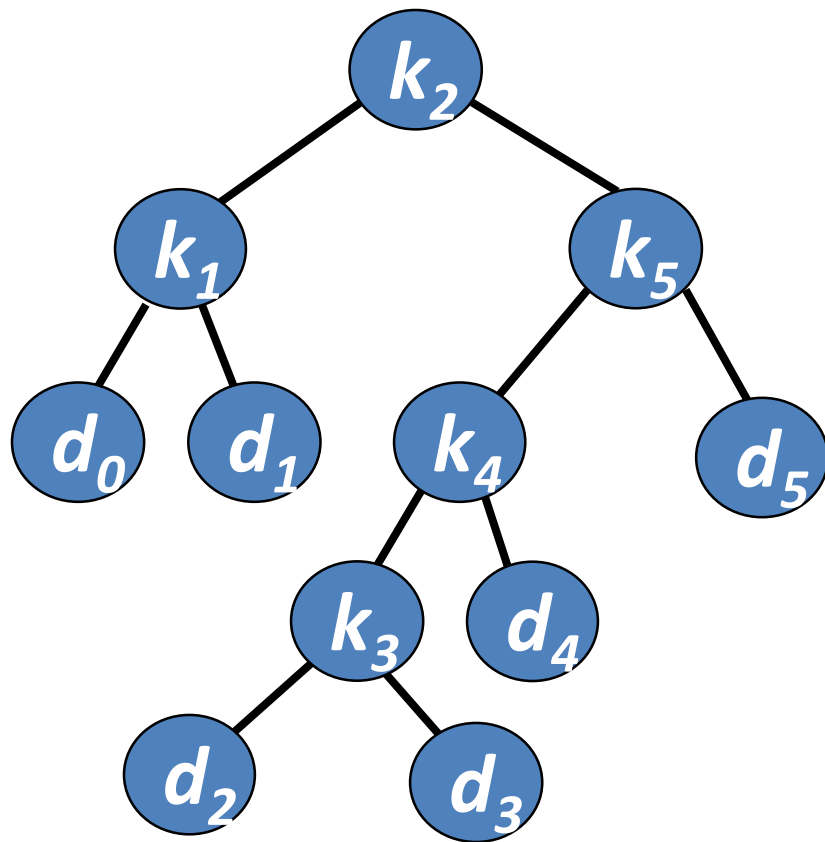
0	1	2	3	4	5
0.05	0.30	0.45	0.55	0.70	1.00
	0.10	0.25	0.35	0.50	0.80
		0.05	0.15	0.30	0.60
			0.05	0.20	0.50
				0.05	0.35
					0.10

(b) $W(i, j)$

$$E(1, 3) = \min \begin{cases} E(1, 0) + E(2, 3) + W(1, 3) \\ = 0.05 + 0.70 + 0.55 = 1.30 \quad \mathbf{r = 1} \\ E(1, 1) + E(3, 3) + W(1, 3) \\ = 0.45 + 0.25 + 0.55 = 1.25 \quad \mathbf{r = 2} \\ E(1, 2) + E(4, 3) + W(1, 3) \\ = 0.90 + 0.05 + 0.55 = 1.50 \quad \mathbf{r = 3} \end{cases}$$

	1	2	3	4	5
1	1	1	2	2	2
2		2	2	2	4
3			3	4	5
4				4	5
5					5

(c) $Root(i, j)$



示例

$$E(i, j) = \begin{cases} q_{i-1} = q_j & \text{if } j = i - 1 \\ \min_{i \leq r \leq j} \{E(i, r-1) + E(r+1, j) + W(i, j)\} & \text{if } j \geq i \end{cases}$$

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

0	1	2	3	4	5
0.05	0.30	0.45	0.55	0.70	1.00
	0.10	0.25	0.35	0.50	0.80
		0.05	0.15	0.30	0.60
			0.05	0.20	0.50
				0.05	0.35
					0.10

$$E(1,3) = \min \begin{cases} E(1,0) + E(2,3) + W(1,3) \\ = 0.05 + 0.70 + 0.55 = 1.30 \quad \mathbf{r = 1} \\ E(1,1) + E(3,3) + W(1,3) \\ = 0.45 + 0.25 + 0.55 = 1.25 \quad \mathbf{r = 2} \\ E(1,2) + E(4,3) + W(1,3) \\ = 0.90 + 0.05 + 0.55 = 1.50 \quad \mathbf{r = 3} \end{cases}$$

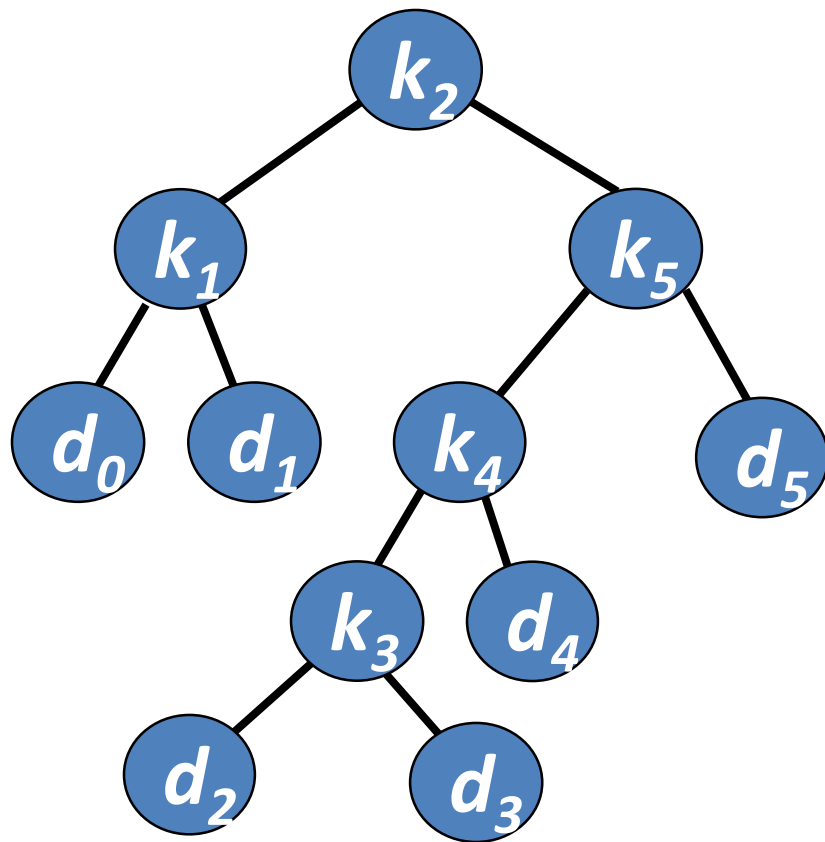
(a) $W(i, j)$

0	1	2	3	4	5
0.05	0.45	0.90	1.25	1.75	2.75
	0.10	0.40	0.70	1.20	2.00
		0.05	0.25	0.60	1.30
			0.05	0.30	0.90
				0.05	0.50
					0.10

(b) $E(i, j)$

1	2	3	4	5
1	1	2	2	2
	2	2	2	4
		3	4	5
			4	5
				5

(c) $Root(i, j)$



DYNAMIC PROGRAMMING



SO BORING!!!